



UNS
UNIVERSIDAD
NACIONAL DEL SANTA

CURSO DE APLICACIONES MÓVILES

Desarrolla. Conecta. Innova.
Lleva tus ideas al siguiente nivel.



Flutter



Kotlin



React Native



Android



iOS

```
import 'package:flutter  
class HomePage extends  
@override  
Widget build(BuildContext  
return Scaffold(  
  appBar: AppBar(  
    body: Center(  
      child: Text(  
        'Hola, Mundo!!  
    )  
  )  
)  
)
```



**DESARROLLO
MULTIPLATAFORMA**
Crea apps para Android
e iOS con un solo código.



**INTERFACES
MODERNAS**
Diseña experiencias
atractivas y funcionales.



**CONECTIVIDAD
Y SERVICIOS**
Integra APIs, bases de datos
y servicios en la nube.



**SEGURIDAD
Y RENDIMIENTO**
Desarrolla aplicaciones
seguras, rápidas y
optimizadas.



**PUBLICACIÓN
Y DESPLIEGUE**
Publica tu app en Google
Play y App Store.



DESARROLLO
DE APPS



UI/UX
MÓVIL



APIS Y
SERVICIOS



BASES DE
DATOS



PRUEBAS Y
DEPLOY



Docente:

Dr. Mirko Manrique Ronceros

INTRODUCCIÓN

El diseño de interfaces en aplicaciones móviles es un aspecto fundamental para garantizar una experiencia de usuario eficiente, intuitiva y atractiva. Una interfaz bien diseñada permite que los usuarios interactúen fácilmente con la aplicación, mejorando la accesibilidad, la navegación y la satisfacción durante el uso del sistema. En el desarrollo móvil, no solo es importante la funcionalidad de la aplicación, sino también la forma en que la información y las herramientas son presentadas al usuario.

Dentro de este contexto, Material Design surge como un sistema de diseño desarrollado por Google que establece principios, componentes visuales y buenas prácticas para la creación de interfaces modernas y consistentes. Este enfoque combina diseño visual, animaciones, tipografía, colores y elementos interactivos con el objetivo de ofrecer aplicaciones más dinámicas, organizadas y fáciles de utilizar.

El estudio de interfaces y Material Design permite comprender cómo estructurar correctamente los elementos gráficos de una aplicación móvil, aplicando estándares actuales de diseño y usabilidad. Asimismo, facilita la creación de aplicaciones visualmente atractivas, adaptables a diferentes dispositivos y orientadas a brindar una mejor experiencia al usuario final.

I. VIEW Y CUSTOMIZE VIEWS

1) Definición de View en Android y su papel en la interfaz de usuario

- **Concepto:** En Android, una View es el bloque de construcción fundamental para cualquier elemento de la interfaz de usuario (UI). Representa un área rectangular en la pantalla y es responsable de dibujar contenido y manejar la interacción del usuario. Casi todo lo que el usuario ve e interactúa en una aplicación Android (textos, botones, imágenes, campos de entrada, etc.) es una instancia de la clase `android.view.View` o de una de sus subclases.
- **Papel en la Interfaz de Usuario:**
 - **Visualización:** Las Views son responsables de dibujar contenido en la pantalla. El sistema Android las organiza en una jerarquía de vistas (View Hierarchy), donde cada View se dibuja dentro de los límites de su padre (ViewGroup).
 - **Interacción:** Las Views son interactivas y pueden responder a eventos de entrada del usuario, como toques, clics, deslizamientos, y eventos del sistema. Proporcionan mecanismos para registrar listeners que se activan cuando ocurren estos eventos.
 - **Presentación de Datos:** Muchas Views se utilizan para mostrar datos al usuario (ej., `TextView` para texto, `ImageView` para imágenes).
 - **Entrada de Datos:** Otras Views permiten al usuario ingresar información (ej., `EditText`).
 - **Control de la Aplicación:** Views como `Button` y `Switch` permiten al usuario interactuar con la lógica de la aplicación.

- **Contenedores:** Aunque técnicamente los ViewGroups (como LinearLayout, ConstraintLayout) son subclases de View, su papel principal es contener y organizar otras Views, definiendo la estructura del layout.
- **Atributos de las Views:** Todas las Views comparten un conjunto común de atributos (definidos en la clase android.view.View) que controlan su apariencia, tamaño, posición, visibilidad, comportamiento, etc. Estos atributos se pueden configurar en el archivo de layout XML o dinámicamente en el código.

2) Creación de Views Personalizadas Mediante la Extensión de Clases

Base:

Android proporciona una amplia gama de Views predefinidas, pero en muchos casos, se necesita crear componentes de UI que tengan un comportamiento o una apariencia específicos. Esto se logra creando Views personalizadas extendiendo las clases base de View o sus subclases.

- **Clases Base para Extender:**
 - **android.view.View:** La clase base para todos los componentes visuales. Se extiende directamente cuando se necesita un control total sobre el dibujo y el comportamiento.
 - **Subclases de View:** Se pueden extender subclases existentes como TextView, ImageView, Button, etc., para modificar su comportamiento o apariencia predeterminados. Esto es útil cuando se necesita una ligera variación de un componente existente.
 - **android.view.ViewGroup:** Si la View personalizada necesita contener y organizar otras Views, se debe extender una subclase de

ViewGroup (como LinearLayout, FrameLayout, o crear una subclase personalizada de ViewGroup).

- **Pasos para Crear una View Personalizada:**

- **Crear una Nueva Clase:** Crear una nueva clase en Java o Kotlin que extienda la clase base apropiada (View o una de sus subclases).
- **Constructores:** Implementar los constructores necesarios. Es importante proporcionar al menos los siguientes constructores para que la View personalizada pueda ser instanciada desde XML y desde código:

Kotlin

constructor(context: Context) : super(context)

constructor(context: Context, attrs: AttributeSet?) : super(context, attrs)

constructor(context: Context, attrs: AttributeSet?, defStyleAttr: Int) : super(context, attrs, defStyleAttr)

constructor(context: Context, attrs: AttributeSet?, defStyleAttr: Int, defStyleRes: Int) : super(context, attrs, defStyleAttr, defStyleRes)

- **context:** El contexto actual.
 - **attrs:** Un conjunto de atributos XML que se especificaron al instanciar la View desde un layout.
 - **defStyleAttr:** Una referencia a un estilo en el tema actual que contiene atributos predeterminados para la View.
 - **defStyleRes:** Una referencia a un estilo predeterminado dentro del tema de la aplicación (solo se usa si no se encuentra defStyleAttr).
- **Sobrescribir onDraw(Canvas canvas):** Este método es donde se realiza el dibujo real de la View. Se utiliza el objeto Canvas

proporcionado para dibujar formas, texto, imágenes, etc., utilizando objetos Paint para definir el color, estilo, fuente, etc.

- **Manejar Atributos Personalizados (Opcional):** Si se desea que la View personalizada tenga atributos específicos que se puedan configurar desde XML, se deben:
 - Declarar los atributos en un archivo `attrs.xml` dentro del directorio `res/values/`.
 - Obtener los valores de estos atributos en el constructor utilizando `context.obtainStyledAttributes(attrs, R.styleable.MiVistaPersonalizada)`.
 - Utilizar los valores obtenidos para configurar la apariencia o el comportamiento de la View.
 - Reciclar el `TypedArray` utilizando `typedArray.recycle()` después de obtener los valores.
- **Manejar Eventos de Entrada (Opcional):** Sobrescribir métodos como `onTouchEvent(MotionEvent event)`, `onKeyDown(int keyCode, KeyEvent event)`, etc., para detectar y responder a las interacciones del usuario.
- **Medir la View (Opcional):** Sobrescribir `onMeasure(int widthMeasureSpec, int heightMeasureSpec)` para especificar el tamaño deseado de la View. Es importante tener en cuenta los "MeasureSpec" proporcionados por el padre, que indican las restricciones de tamaño.

3) Manejo de Eventos y Atributos Personalizados en Views

- **Manejo de Eventos:**

- **onTouchEvent(MotionEvent event):** Proporciona información detallada sobre los eventos táctiles (presionar, mover, levantar). Se puede sobrescribir para implementar gestos personalizados o interacciones táctiles específicas.
- **onClickListener:** Un listener estándar para eventos de clic. Se puede establecer utilizando `setOnClickListener()`.
- **Otros Listeners:** Dependiendo de la necesidad, se pueden usar otros listeners para eventos como toques largos (`OnLongClickListener`), eventos de foco (`OnFocusChangeListener`), eventos de teclado (`OnKeyListener`), etc.

- **Manejo de Atributos Personalizados:**

- **Declaración en attrs.xml:**

XML

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <declare-styleable name="MiVistaPersonalizada">
        <attr name="colorDeFondo" format="color" />
        <attr name="texto" format="string" />
        <attr name="tamanoTexto" format="dimension" />
    </declare-styleable>
</resources>
```

- **Obtención de Atributos en el Constructor:**

Kotlin

```
class MiVistaPersonalizada(context: Context, attrs: AttributeSet?) :
    View(context, attrs) {
```



```

private var colorFondo: Int = Color.RED
private var textoAMostrar: String = ""
private var tamanoTexto: Float = 12f
private val paint = Paint(Paint.ANTI_ALIAS_FLAG)
init {
    attrs?.let {
        val typedArray = context.obtainStyledAttributes(it,
R.styleable.MiVistaPersonalizada)

        colorFondo =
typedArray.getColor(R.styleable.MiVistaPersonalizada_colorDeFon
do, Color.RED)

        textoAMostrar =
typedArray.getString(R.styleable.MiVistaPersonalizada_texto) ?: ""

        tamanoTexto =
typedArray.getDimension(R.styleable.MiVistaPersonalizada_taman
oTexto, 12f)

        typedArray.recycle()
    }
    paint.color = colorFondo
    paint.textSize = tamanoTexto
}
override fun onDraw(canvas: Canvas) {
    super.onDraw(canvas)
    canvas.drawRect(0f, 0f, width.toFloat(), height.toFloat(), paint)
    paint.color = Color.WHITE
    canvas.drawText(textoAMostrar, 10f, height / 2f + tamanoTexto /
2f, paint)
}

```



```
}
```

– **Uso en el Layout XML:**

```
<com.example.app.MiVistaPersonalizada
```

```
    android:layout_width="200dp"
```

```
    android:layout_height="100dp"
```

```
    app:colorDeFondo="#00FF00"
```

```
    app:texto="¡Hola!"
```

```
    app:tamanoTexto="20sp" />
```

- (Recuerda añadir el namespace `xmlns:app="http://schemas.android.com/apk/res-auto"` en el layout raíz).

4) Ejemplos de Implementación de Views Personalizadas para Componentes Específicos

a) Indicador de Progreso Circular Personalizado: Se puede crear una View que dibuje un círculo de progreso animado, con control sobre el color, grosor y animación:

Kotlin

```
class CircularProgressBar(context: Context, attrs: AttributeSet?) :
```

```
View(context, attrs) {
```

```
    // ... (implementación para dibujar el círculo, el arco de progreso y la animación)
```

```
    override fun onDraw(canvas: Canvas) {
```

```
        // ... (dibujo utilizando Paint y Canvas)
```

```
    }
```

```
}
```

- b) Botón con Icono y Texto:** Se puede extender View o LinearLayout para crear un botón que combine un icono (dibujado o desde un recurso) y texto, con control sobre la posición del icono.

Kotlin

```
class IconButton(context: Context, attrs: AttributeSet?) :
    LinearLayout(context, attrs) {
    private val imageView: ImageView
    private val textView: TextView

    init {
        orientation = HORIZONTAL
        // ... (inflar layout o crear ImageView y TextView
        // programáticamente)
        // ... (obtener y aplicar atributos personalizados para el icono y el
        // texto)
    }
}
```

- c) Gráfico de Líneas Personalizado:** Se puede crear una View que dibuje un gráfico de líneas basado en un conjunto de datos, con control sobre los colores de las líneas, etiquetas de los ejes, etc.

Kotlin

```
class LineChartView(context: Context, attrs: AttributeSet?) :
    View(context, attrs) {
    private var dataPoints: List<Float> = emptyList()
    private val linePaint = Paint(Paint.ANTI_ALIAS_FLAG)
    private val textPaint = Paint(Paint.ANTI_ALIAS_FLAG)
```

```

fun setData(data: List<Float>) {
    dataPoints = data
    invalidate() // Forzar un redibujo
}

override fun onDraw(canvas: Canvas) {
    // ... (lógica para dibujar los ejes, las líneas y las etiquetas)
}
}

```

d) Selector de Color Personalizado: Una View que permite al usuario seleccionar un color a través de un círculo de colores o un selector de espectro.

Kotlin

```

class ColorPickerView(context: Context, attrs: AttributeSet?) :
    View(context, attrs) {
    // ... (lógica para dibujar el selector de color y manejar los eventos
    táctiles para la selección)

    override fun onTouchEvent(event: MotionEvent): Boolean {
        // ... (detectar la selección del color)
        return true
    }

    override fun onDraw(canvas: Canvas) {
        // ... (dibujar el selector de color)
    }
}

```

II. MATERIAL DESIGN

1) Introducción

Material Design es un sistema de diseño visual creado por Google para desarrollar interfaces modernas, intuitivas, consistentes y adaptables en aplicaciones móviles, web y multiplataforma.

Fue presentado oficialmente en 2014 y posteriormente evolucionó hacia:

- Material Design 2
- Material You (Material Design 3)

Actualmente, Android utiliza principalmente Material Design 3, el cual incorpora:

- Personalización dinámica.
- Adaptabilidad.
- Mejor accesibilidad.
- Diseño responsivo.
- Experiencia visual moderna.

2) ¿Qué es Material Design?

Material Design es una guía de diseño que define:

- Cómo deben verse las interfaces.
- Cómo deben comportarse los componentes.
- Cómo interactúa el usuario con la aplicación.
- Cómo mantener consistencia visual y experiencia de usuario.

Google lo define como:

“Un sistema adaptable de directrices, componentes y herramientas que respaldan las mejores prácticas de diseño de interfaces de usuario.”

3) Objetivos de Material Design

3.1. Mejorar la experiencia del usuario (UX)

Busca que las aplicaciones sean:

- Fáciles de usar.
- Intuitivas.
- Visualmente agradables.
- Accesibles.

3.2. Mantener consistencia

Todas las aplicaciones Android deben compartir:

- Estilos visuales.
- Comportamientos similares.
- Navegación uniforme.

Esto reduce la curva de aprendizaje del usuario.

3.3. Facilitar el desarrollo

Material Design proporciona:

- Componentes prediseñados.
- Guías oficiales.
- Librerías listas para usar.

3.4. Adaptabilidad

Permite que la interfaz:

- Se adapte a distintos tamaños de pantalla.
- Funcione en teléfonos, tablets y foldables.
- Sea responsiva.

4) Evolución de Material Design

4.1. Material Design 1 (2014)

Características:

- Sombras.
- Elevaciones.
- Capas tipo papel.
- Animaciones fluidas.

4.2. Material Design 2

Introdujo:

- Mayor personalización.
- Bordes redondeados.
- Nuevos estilos tipográficos.

4.3. Material Design 3 (Material You)

Características modernas:

- Colores dinámicos.
- Personalización automática.
- Mejor accesibilidad.
- Componentes inteligentes.

5) Principios Fundamentales de Material Design

5.1. Material como metáfora

La interfaz se comporta como si estuviera hecha de:

- Papel.
- Tinta.
- Superficies físicas.

Cada componente tiene:

- Profundidad.
- Elevación.
- Movimiento.

Ejemplo:

Un botón elevado proyecta sombra.

5.2. Movimiento significativo

Las animaciones:

- No son decorativas.
- Ayudan al usuario a comprender transiciones.

Ejemplos:

- Apertura de pantallas.
- Expansión de tarjetas.
- Aparición de menús.

5.3. Diseño adaptable

La interfaz debe:

- Ajustarse automáticamente.
- Mantener coherencia en cualquier dispositivo.

5.4. Jerarquía visual

Se utiliza:

- Tamaño.
- Color.
- Elevación.
- Espaciado.

Para indicar:

- Qué es más importante.
- Qué debe observar primero el usuario.

6) Componentes Principales de Material Design

6.1. App Bar (Toolbar)

Barra superior que contiene:

- Título.
- Navegación.
- Acciones.

Ejemplo:

- Botón de búsqueda.
- Menú de opciones.

Código XML:

```
<com.google.android.material.appbar.MaterialToolbar  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:title="Mi Aplicación"/>
```

6.2. Floating Action Button (FAB)

Botón flotante circular para acciones principales.

Ejemplo:

- Agregar contacto.
- Crear mensaje.

Código:

```
<com.google.android.material.floatingactionbutton.FloatingActionB  
utton  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"/>
```

6.3. Cards (Tarjetas)

Permiten agrupar información visualmente.

Usos:

- Noticias.
- Productos.
- Eventos.

Código:

```
<com.google.android.material.card.MaterialCardView  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"/>
```

6.4. Navigation Drawer

Menú lateral de navegación.

Usado para:

Navegar entre secciones.

Ejemplo:

- Inicio.
- Perfil.
- Configuración.

6.5. Bottom Navigation

Barra inferior para navegación rápida.

Ideal para:

Apps con 3 a 5 secciones principales.

Ejemplo:

WhatsApp.

Instagram.

6.6. SnackBar

Mensaje temporal mostrado en la parte inferior.

Ejemplo:

```
SnackBar.make(view, "Datos guardados",  
SnackBar.LENGTH_LONG).show()
```

6.7. TextInputLayout

Campos de texto modernos.

Incluyen:

- Validación.
- Etiquetas flotantes.
- Iconos.

6.8. Dialogs

Ventanas emergentes para:

- Confirmaciones.
- Alertas.
- Formularios.

7) Sistema de Colores

Material Design define:

- Primary Color.
- Secondary Color.
- Background.
- Surface.
- Error Colors.

7.1. Colores dinámicos

Material You puede generar colores automáticamente basados en:

- El wallpaper del dispositivo.

7.2. Importancia de la psicología del color

Ejemplos:

Azul → confianza.

Verde → éxito.

Rojo → error/peligro.

8) Tipografía en Material Design

Google recomienda:

- Claridad.
- Legibilidad.
- Escalabilidad.

Fuentes comunes:

- Roboto.
- Google Sans.

8.1. Jerarquía tipográfica

Incluye:

Headline.

Title.

Body.

Label.

9) Elevación y Sombras

La elevación representa profundidad.

Ejemplo:

Un botón flotante tiene más elevación que una tarjeta.

La elevación ayuda a:

- Diferenciar componentes.
- Crear jerarquía visual.

10) Motion Design (Animaciones)

Las animaciones:

- Mejoran UX.
- Guían la atención del usuario.

Ejemplos:

- Fade.
- Transformaciones.
- Shared Element Transitions.

11) Material Design y Accesibilidad

Material Design promueve:

- Contraste adecuado.
- Tamaños legibles.
- Navegación accesible.
- Compatibilidad con lectores de pantalla.

12) Material Design en Android Studio

Dependencia Material

implementation 'com.google.android.material:material:1.12.0'

13) Material Design con Jetpack Compose

Jetpack Compose utiliza Material Design 3 de forma nativa.

Ejemplo:

```
MaterialTheme {
    Text("Hola Mundo")
}
```

14) Ventajas de Material Design

Ventaja	Descripción
Consistencia	Interfaces uniformes
UX mejorada	Mayor facilidad de uso
Adaptabilidad	Responsive
Desarrollo rápido	Componentes listos
Accesibilidad	Inclusión
Modernidad	Interfaces profesionales

15) Desventajas

Desventaja	Descripción
Dependencia visual	Muchas apps pueden parecer similares
Curva de aprendizaje	Requiere comprender guías
Peso adicional	Más librerías

16) Buenas Prácticas

16.1. No saturar colores

Usar:

1 color principal.

1 secundario.

16.2. Mantener consistencia

Todos los botones deben comportarse igual.

16.3. Usar espacios adecuados

Evitar interfaces sobrecargadas.

16.4. Priorizar accesibilidad

Tamaños adecuados.

Contraste suficiente.

17) Comparación entre UI Tradicional y Material Design

UI Tradicional	Material Design
Interfaces rígidas	Interfaces dinámicas
Poco movimiento	Animaciones fluidas
Menor accesibilidad	Alta accesibilidad
Diseño inconsistente	Diseño uniforme

18) Material Design y UX

Material Design está estrechamente relacionado con:

- UX (User Experience).
- UI (User Interface).

Una buena implementación mejora:

- Satisfacción.
- Productividad.
- Retención de usuarios.

19) Conclusiones

- Material Design es el estándar moderno de diseño Android.
- Mejora significativamente la experiencia del usuario.
- Facilita el desarrollo profesional de aplicaciones móviles.
- Material Design 3 incorpora personalización dinámica y accesibilidad avanzada.
- Es fundamental para cualquier desarrollador Android moderno dominar:
 - Material Components.
 - Material You.
 - Jetpack Compose.
 - Responsive Design.